

.NET 10 App Dev Hands-On Lab

EF Lab 5 - Repositories

This creates repositories and interfaces for the data access library. You must have completed EF Lab 4 before starting this lab.

Part 1: Create the Base Repositories

Step 1: Create the BaseView and BaseRepo Interfaces

- Create a new folder in the AutoLot.Dal project named Repos. Create two subfolders under that directory named Base and Interfaces. Add a new folder named Base to the Interfaces folder, and add a new interface named IBaseViewRepo.cs. Update the code for the IBaseViewRepo.cs interface to the following:

```
namespace AutoLot.Dal.Repos.Interfaces.Base;
public interface IBaseViewRepo<T> where T : class
{
    IQueryable<T> ExecuteSqlString(string sql);
    // Returns a materialized list; safe to use after the DbContext is disposed.
    List<T> GetAllAsList();
    List<T> GetAllIgnoreQueryFiltersAsList();
    List<T> GetAllIgnoreQueryFiltersAsList(IEnumerable<string> filtersToIgnore);
    // Returns an IQueryable that defers execution – the DbContext must remain alive
    // until the query is materialized (e.g., .ToList(), iteration, or JSON serialization).
    IQueryable<T> GetAllAsQueryable();
    IQueryable<T> GetAllIgnoreQueryFiltersAsQueryable();
    IQueryable<T> GetAllIgnoreQueryFiltersAsQueryable(IEnumerable<string> filtersToIgnore);
}
```

- Add a new interface named IBaseRepo.cs in the same location and update it to the following:

```
namespace AutoLot.Dal.Repos.Interfaces.Base;
public interface IBaseRepo<T> : IBaseViewRepo<T> where T : BaseEntity, new()
{
    T Find(int? id);
    T FindAsNoTracking(int id);
    T FindIgnoreQueryFilters(int id);
    void ExecuteParameterizedQuery(string sql, object[] sqlParametersObjects);
    int Add(T entity, bool persist = true);
    int AddRange(IList<T> entities, bool persist = true);
    int Update(T entity, bool persist = true);
    int UpdateRange(IList<T> entities, bool persist = true);
    int Delete(int id, long timeStamp, bool persist = true);
    int Delete(T entity, bool persist = true);
    int DeleteRange(IList<T> entities, bool persist = true);
    int ExecuteBulkUpdate(Expression<Func<T, bool>> whereClause,
        Action<UpdateSettersBuilder<T>> setPropertyCalls);
    int ExecuteBulkDelete(Expression<Func<T, bool>> whereClause);
    int SaveChanges();
}
```

- Add the following global using statements to the GlobalUsings.cs file:

```
global using AutoLot.Dal.Repos;
global using AutoLot.Dal.Repos.Interfaces;
global using AutoLot.Dal.Repos.Interfaces.Base;
```

Step 2: Create the BaseView Repository

- Add a new class to the Repos/Base folder named BaseViewRepo.cs, and update the code to the following:

```
namespace AutoLot.Dal.Repos.Base;

public abstract class BaseViewRepo<T>(
    ApplicationDbContext context) : IBaseViewRepo<T> where T : class
{
    private readonly bool _disposeContext = false;
    protected DbSet<T> Table { get; } = context.Set<T>();
    protected ApplicationDbContext Context { get; } = context;
    protected BaseViewRepo(DbContextOptions<ApplicationDbContext> options)
        : this(new ApplicationDbContext(options))
        => _disposeContext = true;
    public IQueryable<T> ExecuteSqlString(string sql) => Table.FromSqlRaw(sql);
    public virtual List<T> GetAllAsList() => GetAllAsQueryable().ToList();
    public virtual List<T> GetAllIgnoreQueryFiltersAsList()
        => GetAllIgnoreQueryFiltersAsQueryable().ToList();
    public virtual List<T> GetAllIgnoreQueryFiltersAsList(IEnumerable<string> filtersToIgnore)
        => GetAllIgnoreQueryFiltersAsQueryable(filtersToIgnore).ToList();
    // The IQueryable overloads below defer execution – the DbContext must remain alive
    // until the query is materialized. Use the AsList variants when the caller does not
    // need to compose additional query operators (filtering, paging, projection).
    public virtual IQueryable<T> GetAllAsQueryable() => Table;
    public virtual IQueryable<T> GetAllIgnoreQueryFiltersAsQueryable()
        => Table.IgnoreQueryFilters();
    public virtual IQueryable<T> GetAllIgnoreQueryFiltersAsQueryable(
        IEnumerable<string> filtersToIgnore) => Table.IgnoreQueryFilters([ .. filtersToIgnore]);
}
```

- Add the following global using statement to the GlobalUsings.cs file:

```
global using AutoLot.Dal.Repos.Base;
```

Step 3: Create the Base Repository

- Add a new class to the Repos/Base folder named BaseRepo.cs and update the code to the following:

```
namespace AutoLot.Dal.Repos.Base;
```

```
public abstract class BaseRepo<T> : BaseViewRepo<T>, IBaseRepo<T> where T : BaseEntity, new()
{
    protected BaseRepo(ApplicationDbContext context) : base(context) {}
    protected BaseRepo(DbContextOptions<ApplicationDbContext> options) : base(options) { }
    public virtual T Find(int? id) => id == null ? null : Table.Find(id.Value);
    public virtual T FindAsNoTracking(int id)
        => Table.AsNoTracking().FirstOrDefault(e => e.Id == id);
    public virtual T FindIgnoreQueryFilters(int id)
        => Table.IgnoreQueryFilters().FirstOrDefault(e => e.Id == id);
    public virtual void ExecuteParameterizedQuery(string sql, object[] sqlParametersObjects)
        => Context.Database.ExecuteSqlRaw(sql, sqlParametersObjects);
    public virtual int Add(T entity, bool persist = true)
    {
        Table.Add(entity);
        return persist ? SaveChanges() : 0;
    }
    public virtual int AddRange(IList<T> entities, bool persist = true)
    {
        Table.AddRange(entities);
        return persist ? SaveChanges() : 0;
    }
    public virtual int Update(T entity, bool persist = true)
    {
        Table.Update(entity);
        return persist ? SaveChanges() : 0;
    }
    public virtual int UpdateRange(IList<T> entities, bool persist = true)
    {
        Table.UpdateRange(entities);
        return persist ? SaveChanges() : 0;
    }
    //This delete shows using entity state to eliminate a query
    public virtual int Delete(int id, long timeStamp, bool persist = true)
    {
        var entity = new T { Id = id, TimeStamp = timeStamp };
        Context.Entry(entity).State = EntityState.Deleted;
        return persist ? SaveChanges() : 0;
    }
    public virtual int Delete(T entity, bool persist = true)
    {
        Table.Remove(entity);
        return persist ? SaveChanges() : 0;
    }
    public virtual int DeleteRange(IList<T> entities, bool persist = true)
    {
        Table.RemoveRange(entities);
        return persist ? SaveChanges() : 0;
    }
}
```

```
public virtual int ExecuteBulkUpdate(
    Expression<Func<T, bool>> whereClause, Action<UpdateSettersBuilder<T>> setPropertyCalls)
    => Table.IgnoreQueryFilters().Where(whereClause).ExecuteUpdate(setPropertyCalls);

public virtual int ExecuteBulkDelete(Expression<Func<T, bool>> whereClause)
    => Table.IgnoreQueryFilters().Where(whereClause).ExecuteDelete();

public virtual int SaveChanges()
{
    try
    {
        return Context.SaveChanges();
    }
    catch (CustomException)
    {
        //Should handle intelligently - already logged
        throw;
    }
    catch (Exception ex)
    {
        //Should log and handle intelligently
        throw new CustomException("An error occurred updating the database", ex);
    }
}
```

Part 2: Add the Entity-Specific Repo Interfaces and Repos

Each model has an interface and a repo that uses the base repository to share functionality.

Step 1: Create the Interface Files

- Create the following files in the Repos\Interfaces folder:

```
//ICarDriverRepo.cs
namespace AutoLot.Dal.Repos.Interfaces;
public interface ICarDriverRepo : IBaseRepo<CarDriver> { }
```

```
//ICarRepo.cs
namespace AutoLot.Dal.Repos.Interfaces;
public interface ICarRepo : IBaseRepo<Car>
{
    List<Car> GetAllByAsList(int makeId);
    // Returns deferred IQueryable – DbContext must remain alive until materialized.
    IQueryable<Car> GetAllByAsQueryable(int makeId);
    string GetPetName(int id);
    int SetAllDrivableCarsColorAndMakeId(string color, int makeId);
    int DeleteNonDrivableCars();
}
```

```
//IDriverRepo.cs
namespace AutoLot.Dal.Repos.Interfaces;
public interface IDriverRepo : IBaseRepo<Driver> { }
```

```
//IMakeRepo.cs
namespace AutoLot.Dal.Repos.Interfaces;
public interface IMakeRepo : IBaseRepo<Make> { }
```

```
//IRadioRepo.cs
namespace AutoLot.Dal.Repos.Interfaces;
public interface IRadioRepo : IBaseRepo<Radio> { }
```

Step 2: Create the MakeRepo Class

- Create a new class named MakeRepo.cs in the Repos directory and update the code to the following:

```
namespace AutoLot.Dal.Repos;
public class MakeRepo : BaseRepo<Make>, IMakeRepo
{
    public MakeRepo(ApplicationDbContext context) : base(context) { }
    internal MakeRepo(DbContextOptions<ApplicationDbContext> options) : base(options) { }
    internal IOrderedQueryable<Make> BuildBaseQuery() => Table.OrderBy(m => m.Name);
    public override IQueryable<Make> GetAllAsQueryable() => BuildBaseQuery();
    public override IQueryable<Make> GetAllIgnoreQueryFiltersAsQueryable()
        => BuildBaseQuery().IgnoreQueryFilters();
    public override IQueryable<Make> GetAllIgnoreQueryFiltersAsQueryable(
        IEnumerable<string> filtersToIgnore)
        => BuildBaseQuery().IgnoreQueryFilters([ .. filtersToIgnore]);
}
```

Step 3: Create the CarDriverRepo Class

- In the Repos folder, create a new class named CarDriverRepo.cs and update the code to the following:

```
namespace AutoLot.Dal.Repos;
public class CarDriverRepo : BaseRepo<CarDriver>, ICarDriverRepo
{
    public CarDriverRepo(ApplicationDbContext context) : base(context) { }
    internal CarDriverRepo(DbContextOptions<ApplicationDbContext> options) : base(options) { }
    internal IQueryable<CarDriver> BuildBaseQuery() =>
        Table.Include(cd => cd.CarNavigation).Include(cd => cd.DriverNavigation);
    public override IQueryable<CarDriver> GetAllAsQueryable() => BuildBaseQuery();
    public override IQueryable<CarDriver> GetAllIgnoreQueryFiltersAsQueryable()
        => BuildBaseQuery().IgnoreQueryFilters();
    public override IQueryable<CarDriver> GetAllIgnoreQueryFiltersAsQueryable(
        IEnumerable<string> filtersToIgnore) =>
        BuildBaseQuery().IgnoreQueryFilters([ .. filtersToIgnore]);
    public override CarDriver Find(int? id) => id == null ? null
        : BuildBaseQuery().IgnoreQueryFilters().FirstOrDefault(cd => cd.Id == id.Value);
    public override CarDriver FindAsNoTracking(int id)
        => BuildBaseQuery().AsNoTracking().FirstOrDefault(cd => cd.Id == id);
    public override CarDriver FindIgnoreQueryFilters(int id)
        => BuildBaseQuery().IgnoreQueryFilters().FirstOrDefault(cd => cd.Id == id);
}
```

Step 4: Create the DriverRepo Class

- Create a new class named DriverRepo.cs in the Repos directory and update the code to the following:

```
namespace AutoLot.Dal.Repos;
public class DriverRepo : BaseRepo<Driver>, IDriverRepo
{
    public DriverRepo(ApplicationDbContext context) : base(context) { }
    internal DriverRepo(DbContextOptions<ApplicationDbContext> options) : base(options) { }
    internal IOrderedQueryable<Driver> BuildBaseQuery()
        => Table.OrderBy(d => d.PersonInformation.LastName)
            .ThenBy(d => d.PersonInformation.FirstName);
    public override IQueryable<Driver> GetAllAsQueryable() => BuildBaseQuery();
    public override IQueryable<Driver> GetAllIgnoreQueryFiltersAsQueryable()
        => BuildBaseQuery().IgnoreQueryFilters();
    public override IQueryable<Driver> GetAllIgnoreQueryFiltersAsQueryable(
        IEnumerable<string> filtersToIgnore) =>
        BuildBaseQuery().IgnoreQueryFilters([ .. filtersToIgnore ]);
}
```

Step 5: Create the RadioRepo Class

- Create a new class named RadioRepo.cs in the Repos directory and update the code to the following:

```
namespace AutoLot.Dal.Repos;
public class RadioRepo : BaseRepo<Radio>, IRadioRepo
{
    public RadioRepo(ApplicationDbContext context) : base(context) { }
    internal RadioRepo(DbContextOptions<ApplicationDbContext> options) : base(options) { }
}
```

Step 6: Create the CarRepo Class

- Create a new class named CarRepo.cs in the Repos directory and update the code to the following:

```
namespace AutoLot.Dal.Repos;
public class CarRepo : BaseRepo<Car>, ICarRepo
{
    public CarRepo(ApplicationDbContext context) : base(context) { }
    internal CarRepo(DbContextOptions<ApplicationDbContext> options) : base(options) { }
    internal IOrderedQueryable<Car> BuildBaseQuery()
        => Table.Include(c => c.MakeNavigation).OrderBy(c => c.PetName);
    public override IQueryable<Car> GetAllAsQueryable() => BuildBaseQuery();
    public override IQueryable<Car> GetAllIgnoreQueryFiltersAsQueryable()
        => BuildBaseQuery().IgnoreQueryFilters();
    public override IQueryable<Car> GetAllIgnoreQueryFiltersAsQueryable(
        IEnumerable<string> filtersToIgnore)
        => BuildBaseQuery().IgnoreQueryFilters([ .. filtersToIgnore ]);
    public override Car Find(int? id)
        => id == null ? null
            : BuildBaseQuery().IgnoreQueryFilters().FirstOrDefault(c => c.Id == id.Value);
    public override Car FindAsNoTracking(int id)
        => BuildBaseQuery().AsNoTracking().FirstOrDefault(c => c.Id == id);
    public override Car FindIgnoreQueryFilters(int id)
        => BuildBaseQuery().IgnoreQueryFilters().FirstOrDefault(c => c.Id == id);
    public List<Car> GetAllByAsList(int makeId) => GetAllByAsQueryable(makeId).ToList();
    public IQueryable<Car> GetAllByAsQueryable(int makeId)
        => BuildBaseQuery().Where(c => c.MakeId == makeId);
    public string GetPetName(int id)
    {
        var outParam = new SqlParameter("@petName", SqlDbType.NVarChar, 50)
            { Direction = ParameterDirection.Output };
        ExecuteParameterizedQuery("EXEC GetPetName @id, @petName OUT",
            [new SqlParameter("@id", id), outParam]);
        return outParam.Value?.ToString();
    }
    public int SetAllDrivableCarsColorAndMakeId(string color, int makeId)
        => ExecuteBulkUpdate(c => c.IsDrivable,
            s => s.SetProperty(c => c.Color, color).SetProperty(c => c.MakeId, makeId));
    public int DeleteNonDrivableCars() => ExecuteBulkDelete(c => !c.IsDrivable);
}
```

Part 3: Create the Extension Method for ASP.NET Core

The following extension method will be used by ASP.NET Core in an upcoming lab.

- Create a new folder named Extensions in the AutoLot.Dal project. In this folder, create a new public static class named ServiceCollectionExtensions.cs and update the code to match the following:

```
namespace AutoLot.Dal.Extensions;
public static class ServiceCollectionExtensions
{
    extension(IServiceCollection services)
    {
        public IServiceCollection AddAutoLotDal(string connectionString)
        {
            services.AddDbContextPool<ApplicationDbContext>(options =>
            {
                options.ConfigureWarnings(wc => wc.Ignore(RelationalEventId.BoolWithDefaultWarning));
                options.UseSqlServer(connectionString, sqlOptions
                    => sqlOptions.EnableRetryOnFailure().CommandTimeout(60));
            });
            services.AddScoped<ICarDriverRepo, CarDriverRepo>();
            services.AddScoped<ICarRepo, CarRepo>();
            services.AddScoped<IDriverRepo, DriverRepo>();
            services.AddScoped<IMakeRepo, MakeRepo>();
            services.AddScoped<IRadioRepo, RadioRepo>();
            return services;
        }
    }
}
```

Summary

The lab created repositories and their interfaces.

Next steps

In the next part of this tutorial series, you will create a data initializer.